Ders 10 Genişletilmiş Notu: Uygulama: Spor Salonu Sistemi Örneğiyle Fizibilite, Gereksinim Analizi ve Sistem Tasarımı Diyagramları

Ders 10 Genişletilmiş Notu: Uygulama: Spor Salonu Sistemi Örneğiyle Fizibilite, Gereksinim Analizi ve Sistem Tasarımı Diyagramları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Problem Tanımı: Spor Salonu Sistemi
- 2. Sistem Analizi ve Tasarımının Adımları
- 3. Fizibilite Çalışması
- 4. Gereksinim Analizi
- 5. Sistem Tasarımı
- 6. Değerlendirme Tablosu
- 7. Özet ve Diyagram Araçları

Bu ders bir uygulama dersidir. Dersi hocanın devrettiği asistan anlattı. Daha önce teoride görülen adımlar, tek bir örnek proje üzerinde baştan sona uygulandı: spor salonu sistemi. Örnek, kendi projelerinizde izleyeceğiniz yolun bir modeli olarak sunuldu.

1. Problem Tanımı: Spor Salonu Sistemi

1.1 Mevcut Durum ve Sorun

Hizmet vermekte olan bir spor salonunun müşteri sayısı artmış, ders içerikleri genişlemiştir. Bu nedenle salon yeni bir sisteme geçmek istemektedir. Mevcut sistemde müşteriler salona giderek kayıt yaptırmakta, kayıtlar yönetim tarafından yerel olarak depolanmaktadır. Bu durum zaman kaybı oluşturmaktadır.

Asistan sorunun nasıl ortaya çıktığına dikkat çekti: Müşteri sayısı azken bu yöntem belki problem yaratmıyordu. Problem, müşteri sayısı arttığında oluştu. Yani yeni sistem ihtiyacını doğuran şey, mevcut yöntemin büyüyen işe yetmemesidir.

1.2 İstenen Sistem

Bizden çevrim içi (online) bir sistem tasarlamamız isteniyor. Sistemde işlemler şöyle yürüyecektir:

- Kişiler salona çevrim içi üye olacak ve derslerini kendileri seçecek.
- Sınıfların takvimi, sınıfta ne zaman ders olduğu ve eğitmen bilgileri görüntülenebilecek.
- Yönetimin belirlediği derslerin tarih ve saat bilgileri ile sınıf kapasiteleri görüntülenebilecek.
- Aerobik, step, pilates gibi özel derslere katılım mümkün olacak, bunlar için ayrı ücret ödenecek.
- Eğitmenler sistemdeki kişisel bilgilerini güncelleyebilecek.

1.3 Ücretlendirme Kuralı

Her ay sabit bir üyelik ücreti alınır. Bu ücret, fitness salonundan süresiz yararlanmayı kapsar. Pilates, dans ve benzeri özel eğitimler ise bu ücrete dahil değildir, bunlar için ekstra ödeme yapılır. Örneğin dans için hem bir grup dersi hem de ayrıca özel ders olabilir. İkisi için de ayrı ücret ödenir.

1.4 Aktörler ve Yapabilecekleri

Projede üç aktör vardır: spor salonu yönetimi, üye ve eğitmen. Sistemi kullanacak kişilerin yapacağı işlemler problem tanımında kısaca özetlenebilir:

Aktör	Yapabileceği işlemler
Spor salonu yönetimi	Üyelik başvurularını onaylar. Derslere ait sınıfları oluşturur. Tarih, saat ve eğitmen bilgilerini girer, gerekirse günceller.
Üye	Bilgilerini girerek üye olur. Sınıfların takvim ve eğitmen bilgilerini görüntüler. Kontenjanı olan sınıflara kaydolur. İsterse eğitmenlerden özel ders talep eder.
Eğitmen	Sistemdeki bilgilerini günceller. Ders verdiği sınıfların katılımcı listelerini görüntüler.

2. Sistem Analizi ve Tasarımının Adımları

Örnek üç adımda ilerler. Önce bir **fizibilite çalışması** yapılır, ardından **gereksinim analizi** gerçekleştirilir, son adımda **sistem tasarımı** yapılır. Asistan, bu adımların ara projelerde ve bitirme projelerinde de aynı şekilde uygulanacağını, dersin bunları hazırlarken yardımcı olmayı amaçladığını söyledi.

3. Fizibilite Çalışması

Fizibilite çalışması altı türe ayrılır: teknik, ekonomik, zaman, sosyal, yönetim ve yasal fizibilite.

3.1 Teknik Fizibilite

Teknik fizibilitede sistemin hayata geçmesinde kullanılacak yazılım ve donanım birimleri araştırılır. Sorulan soru şudur: Bu proje için hangi donanımlara ihtiyacım var, hangi yazılımı kullanmam gerekiyor?

Yazılım. Örnekte sistemin nesneye dayalı programlamayla geliştirileceği, ön ve arka plan geliştirmesinin Java ve JavaScript ile yapılacağı varsayıldı. Bu bir varsayımdır ve değişebilir. C#, Python ve Django gibi farklı seçenekler de kullanılabilir.

Donanım. Spor salonu donanım olarak yetersizdir, eski bilgisayarlara sahiptir. Yeni bilgisayarlara, sunucuya (server) ve yönlendiriciye (router) ihtiyaç duyulur. Çevrim içi sisteme geçildiğinde kullanıcı bilgilerini tutmak için zaten bir sunucu gerekir. Slaytta donanım birimi olarak masaüstü bilgisayar ve kablosuz (wireless) router da belirtilmiştir.

Bu aşamada tedarik edilebilecek donanım birimleri teknik özellikleriyle birlikte yazılır. Çünkü ekonomik fizibilite bu listeye göre oluşturulacaktır. Daha iyi donanım kullanmak ya da donanım sayısını artırmak, projenin maliyetini de artırır. Listedeki elemanlar projeye bakış açınıza ve projeden isteğinize göre değişebilir.

Ölçeklenme alternatifleri. Kullanıcı sayısı arttığında veri tabanı için ayrı bir sunucu alınabilir. Sonraki aşamada veri tabanı ve uygulama sunucusu bulut (cloud) ortamına taşınabilir. Veri tabanını ayrı sunucuda ya da bulutta tutmak ayrı bir maliyet getirir. Bu alternatifler çoğaltılabilir. Asistan bu noktada sınıftan başka örnekler istedi, yanıt gelmedi.

3.2 Ekonomik Fizibilite

Ekonomik fizibilitede yazılım ve donanım masrafları çıkarılır. İki soru cevaplanmaya çalışılır: Bu projenin maliyeti bana ne kadar olacak? Bu proje ne kadar sürede başa baş noktasına ulaşacak, yani ne zaman kâra geçebileceğim? Bunların tahminleri yapılır.

Bu hesap daha projeye başlamadan fikir verir. Ne kadar harcayacağınızı görerek başlarsınız. Kalemler şunlardır: yazılım şirketine ödenecek ücret, bilgisayarların, router'ların ve sunucunun maliyeti, yazılım araçları. Örnekte yazılım araçları ücretsiz alındı, ama ücretli bir araç da kullanılabilirdi. Asistan slayttaki ücretlerin yalnızca sembolik olduğunu belirtti.

> **Ek bilgi (derste anlatılmadı):** Başa baş noktası, birikimli gelirin birikimli maliyete eşit olduğu andır. Bu andan önce proje zararda, sonrasında kârdadır.

3.3 Zaman Fizibilitesi

Zaman fizibilitesi, sistemin ne kadar sürede planlanıp hayata geçirileceğinin ve iş akışının tarihlere göre nasıl olacağının belirlendiği adımdır.

Gantt çizelgesi. Zaman planlaması Gantt çizelgesiyle yapılır. Bu çizelge önceki uygulama dersinde anlatılmıştı. Planlamada hangi işi hangi kaynağın yapacağı atanır. Kaynakların saat başına alacağı ücretler de görülebilir. Böylece projenin genel olarak nasıl işleyeceği ve ne zaman biteceği çizelge üzerinden görülür.

Gantt çizelgesinin yanı sıra süreçleri görmek için PERT diyagramı da kullanılabilir. Asistan, Gantt çizelgesinin PERT'e göre daha net bir görüntüsü olduğunu, iş adımlarının ilişkilerini daha net gösterdiğini ve genellikle PERT yerine Gantt'ın kullanıldığını söyledi.

> **Ders sonrası not:** Asistan iş adımlarının ilişkilerini Gantt çizelgesinin PERT'ten daha net gösterdiğini söyledi; doğrusu tersidir. Gantt çizelgesi işleri zaman ekseninde çubuklarla gösterdiği için süre ve takvimi iyi anlatır. İşler arasındaki bağımlılıkları ve kritik yolu ise PERT/CPM ağ diyagramı daha açık gösterir.

Soru: Süreç Gantt çizelgesinde belirtildiği gibi işlemezse bunu nasıl belirtmeliyiz? Sözlü olarak "böyle planladık ama yapamadık" mı demeliyiz, yoksa başka bir yöntemi var mı?

Cevap: Bu, kullandığınız yazılım modeline göre değişebilir. Bazı modellerde iş uzadığında geri dönüş sağlayabilirsiniz. Süreler projede net olarak belirlendiyse (15 gün ya da 6 gün gibi) uzama ara toplantılarda belirtilebilir. Burada iki durum ayrılır:

- **Esneklik payı olan iş:** İşin örneğin iki gün esneklik payı varsa, bu pay içindeki uzama sıkıntı yaratmaz.
- **Esneklik payı olmayan iş:** Uzama doğrudan diğer işleri de etkiler.

Her iki durumda da konu toplantıda belirlenir ve Gantt çizelgesi güncellenir. Asistanın MS Project örneği şöyleydi: Ayrıntılı tasarım işi bitirilemedi ve dört günden beş güne çıktı. Bu, tasarım birim toplantısında belirtilir. MS Project'te süre 5 olarak girilince gün sayısı artar ve program sonraki işlerin tarihlerini ona göre kendisi ayarlar.

Asistanın iki sayısını birlikte ele alırsak (ara adım eklendi): Gecikme $5-4=1$ gündür. Sonraki işlerdeki kayma $\max(0,\ \text{gecikme}-\text{esneklik payı})$ ile bulunur. İki gün esneklik payında kayma $\max(0,\ 1-2)=0$ gün, esneklik payı olmayan işte $\max(0,\ 1-0)=1$ gündür.

Asistan ayrıca, küçük projelerde işlerin fazla sarkmayabileceğini, bu tür uzamaların daha çok büyük projelerde bir aksaklık nedeniyle ortaya çıkabileceğini ekledi. Örneğin "Yazılımcı 2"nin işi uzadığında, test uzmanının da bunu takip edebilmesi için çizelge yeniden düzenlenir. Büyük projelerde raporlar ortak ilerler ve herkese açık olur. Bütün ekip değişiklikten haberdar olur.

Yazılım geliştirme modelleri. Seçilen yazılım geliştirme modeli zaman fizibilitesini etkiler. Örnek projede şelale (waterfall) modeli kullanılıyor.

Şelale modeli, daha önce benzer projeler gerçekleştirildiyse kullanıma uygundur. Daha önce benzeri yapılmamış, farklı bir projeye giriliyorsa çok uygun değildir, çünkü **şelale modelinde geri besleme yoktur**. Asistan bunu şöyle somutlaştırdı: Analizi yaptım, tasarımı gerçekleştirdim, gerçekleme aşamasında bir problem olduğunu düşündüm. Tasarım aşamasına dönüp

onu tekrarlayıp yeniden gerçekleme, test ve bakım aşamasına gelme gibi bir adımım yok. Şelalede analizin tamamen bittiği, tasarımın tamamen doğru tamamlandığı varsayılarak ilerlenir. Asistana göre bu yüzden şelale modeli büyük ölçekli projelerde kullanıma uygun değildir.

> **Hocanın vurgusu:** Her projeye uygun yazılım modeli farklıdır. Projenin başarıya ulaşması için modeli projeye göre doğru seçmek gerekir.

Örnek projede şelale tercih edildi, ama alternatif olarak yinelemeli (iteratif) şelale modeli de seçilebilirdi. Bu modelde geri besleme vardır: Tasarımı yaptım, gerçekleme adımına geldim, tasarımda bir sıkıntı olduğunu gördüm. Tasarım adımına geri dönerim, eksikliği veya hatayı düzeltirim, sonra yeniden gerçekleme adımına dönerim.

> **Ders sonrası not:** Asistan şelalenin büyük ölçekli projelere uygun olmadığını söyledi. Belirleyici olan büyük olasılıkla yalnız ölçek değil, gereksinimlerin kararlılığıdır: Şelale, gereksinimleri baştan net olan projelere daha uygundur. Asistanın "daha önce benzeri yapıldıysa uygun" gerekçesi de bu yöne işaret eder.

3.4 Sosyal Fizibilite

Sosyal fizibilitede şu sorulur: Yapacağım proje toplum tarafından kabul görecek mi? Bir şey için efor sarf edeceğim, bunun bana bir maliyeti olacak, peki getirisi olacak mı?

Örnekte, planlanan türden çevrim içi üyelik içeren sistemlerin farklı spor salonlarında zaten kullanıldığı görülüyor. Artık birçok işlem çevrim içi yapılabiliyor: birçok yere üye olunabiliyor, banka işlemleri yürütülebiliyor. Bu işlemleri yapan yaş aralığı da genişledi. Spora çok yönelen genç nesiller aynı zamanda internet, telefon ve sosyal medyayla iç içe. Sonuç olarak sistemin kullanılabileceği ve getiri sağlayabileceği düşünülüyor.

3.5 Yönetim Fizibilitesi

Ekiple geliştirilecek sistemin yönetim tarafından istenmesi ve desteklenmesi gerekir. Yönetimin desteklemediği bir projenin hayata geçmesi çok zordur.

3.6 Yasal Fizibilite

Kişilerin sistemde paylaştığı bilgiler, kişisel verilerin korunması kanunu gereği koruma altına alınır. Bu durum üyelik aşamasında belirtilir. Yani kullanıcılar sisteme üye olurken verilerinin korunacağını onlara bildirmek gerekir.

3.7 Fizibilite Matrisi

Fizibilite matrisi, önerilen sistemleri altı fizibilite türü üzerinden karşılaştırır ve projeye uygun olanın seçilmesini sağlar. Örnekte üç sistem önerildi, ama bunun bir sınırı yoktur. Dört, beş, altı sistem de önerilebilir.

Önerilen sistemleri çoğaltmakta fayda vardır. Tek bir sistem görülürse karşılaştırma yapılamayabilir. Hangisinin projeye daha uygun olduğu, ancak farklı sistemler görülerek anlaşılır.

	Sistem 1	Sistem 2	Sistem 3
Teknik	Java, JavaScript, 1 server	C#, .NET, 2 server	Python, Django, cloud, güvenlik hizmeti
Ekonomik	20.300 USD	25.300 USD	35.000 USD
Zaman	27 gün	30 gün	25 gün
Sosyal	Kabul edilebilir	Kabul edilebilir	Kabul edilebilir
Yönetim	Sıkıntı yok	Sıkıntı yok	Sıkıntı yok
Yasal	KVKK	KVKK	KVKK + güvenlik hizmeti

Asistan seçimi şöyle gerekçelendirdi. Sistem 3'te güvenlik hizmeti verilmesi ve bulutun kullanılması avantajlı görünür, ama maliyeti daha yüksektir. "Bu maliyeti harcayacağım, peki

getirisi ne kadar olacak, ne kadar kâr edeceğim?” diye karşılaştırınca, şu anki işletme için maliyet yüksek bulunur. Sistem 2’de de iki sunucu vardır ve çevrim içi sisteme geçiş aşamasında maliyeti yüksektir. Başlangıç için işleri tek bir sunucu görebilir. Bu yüzden örnekte **Sistem 1** seçildi. Asistan dördüncü bir sistem önerisi de sordu, yanıt gelmedi.

> **Ders sonrası not:** Matristeki sayılarla Sistem 1’e göre farklar şöyledir. Sistem 2: $25.300 - 20.300 = 5.000$ USD daha pahalı, $30 - 27 = 3$ gün daha uzun. Sistem 3: $35.000 - 20.300 = 14.700$ USD daha pahalı, $27 - 25 = 2$ gün daha kısa. Sistem 2 hem maliyette hem sürede Sistem 1’in gerisindedir. Sistem 3 ise gerçek bir ödünleşimdir: 2 gün kazanmanın ve bulut ile güvenlik hizmetinin bedeli 14.700 USD’dir. Sistem 1, en kısa süreli seçenek değildir; seçim bu nedenle maliyet ve başlangıç ihtiyacı gerekçesine dayanır.

Asistan matrisin başka bir özelliğine de dikkat çekti: Önerilen sisteme göre fizibilite türleri farklılık gösterebilir. Bu örnekte sosyal ve yönetim fizibilitesi sistemler arasında değişmedi, ama yasal ve diğerleri değişti.

3.8 Neden Nesneye Yönelik Bir Dil?

Önerilen sistemde teknik fizibilite kısmında, nesneye yönelik programlamayı destekleyen ve web destekli herhangi bir dil tercih edilebilir. Örnekte Java seçildi. Asistan bu tercihin nedenini sordu, yanıt gelmeyince slayttaki gerekçeleri açıkladı ve bunların çoğaltılabileceğini söyledi:

- Sınıflar sayesinde bütün projede değişiklik yapmak yerine tek bir sınıfta değişiklik yapılıp projenin çalışması sağlanabilir.
- Sınıflar bir kez oluşturulduğu için uzun kod tekrarlarından kurtulunur.
- Nesneler birbirinden bağımsız olduğu için “bilgi gizliliği” konusunda avantaj sağlanır.

  > **Ders sonrası not:** Slayttaki “bilgi gizliliği” ifadesinin doğru terimi bilgi gizlemedir (information hiding). Bu, sınıfın iç ayrıntılarının `private` gibi erişim belirleyicileriyle dışarıdan saklanmasıdır. Veri güvenliği ya da mahremiyet anlamında bir gizlilik değildir.

4. Gereksinim Analizi

Ön inceleme yapıldıktan ve fizibilite adımlarında elde edilen raporlar incelendikten sonra gereksinim analizine geçilir. Bu adımda ayrıntılı görüşmeler sonucunda sistemin ihtiyaçları belirlenir ve modellemeler yapılır. Tasarım aşamasının temelleri burada oluşturulur.

4.1 Veri Toplama: Anket ve Görüşme

Veri toplama yöntemi olarak kullanıcılara açık uçlu ve kapalı uçlu sorulardan oluşan bir anket sunulur. Spor salonu kullanıcılarına sorular sorarak geliştirilecek proje yönlendirilmeye çalışılır.

Asistan bunu bir de yazılım şirketi gözüyle anlattı: Size bir müşteri gelir. Müşterinin isteklerini dinlediğiniz safhadır bu ve tasarımı artık bu isteklere göre modellemeye başlarsınız. Örnekte de kullanıcılarla görüşme ve anket yoluyla veri toplanıyor. Anketteki örnek soru “Var olan sistemde sorun yaşıyor musunuz?” şeklindedir. Proje verilen cevaplara göre şekillendirilir.

4.2 Kullanım Senaryosu (Use Case)

Kullanım senaryosu, aktörlerin proje üzerinde yapabileceği işlemleri anlatır. Aktörler ilk safhada belirlenmişti: spor salonu yönetimi, eğitmen ve üye.

Kullanım senaryosu dokümanı. Diyagram çizilmeden önce kullanım senaryosu dokümantasyonu oluşturulur. Derste örnek olarak ödeme işlemi ele alındı:

Alan	Ödeme senaryosu
Birincil aktör	Üye (ödeme işlemini gerçekleştirecek kişi)

Alan	Ödeme senaryosu
İlgiler ve beklentiler	Üyeler doğru hesaplanan ücreti çevrim içi sistem üzerinden sorunsuz ödemek ister.
Ön koşul	Kişi üye olmalı, sistemde çevrim içi bulunmalıdır.
Son koşul (slaytta)	Ödeme bilgileri doğru olmalıdır.
Ana akış	1) Üye ödenecek toplam tutarı görüntüler. 2) Bu tutar için kredi kartı ve fatura bilgilerini girer. 3) Üyenin onayıyla ödeme gerçekleşir.
Alternatif akış	Üyenin verdiği bilgilerde yanlışlık varsa sistem hata mesajı verir ve bilgileri yeniden girmesini ister.

Ana akış, olması beklenen akıştır. Bunun yanında alternatif akış da yazılır. Bu doküman yalnız ödeme işlemi içindir. Geri kalan bütün işlemler için de aynı şekilde kullanım senaryosu dokümanı hazırlanır.

> **Ders sonrası not:** Slaytta son koşul "ödeme bilgileri doğru olmalıdır" diye geçiyor; bu cümle büyük olasılıkla bir ön koşul veya doğrulama kuralı olarak düşünülmelidir. Son koşul, senaryo başarıyla bitince sistemin hangi durumda olduğunu anlatır. Bu senaryo için örneğin "ödeme kaydedilir, üyenin borcu güncellenir" yazılabilir.

Kullanım senaryosu diyagramı. Diyagramın gösterim kuralları şunlardır:

- **Sistem:** Dikdörtgenle gösterilir. Yapılabilecek işlemler bu dikdörtgenin içindedir. Örnekte dikdörtgen spor salonu sistemidir.
- **Aktör:** Çöp adamla gösterilir.
- **Senaryo (işlem):** Elipsle gösterilir.
- **include:** Bir işlem başka bir işlemi de içeriyorsa kullanılır. Üyenin sınıfa kaydolması, sınıf bilgisini görüntüleme işlemini de içerdiği için include ile ifade edilir.
- **extend:** İsteğe bağlı olan işlem için kullanılır. Üye sınıfa kaydolurken veya kaydolduktan sonra isterse özel ders seçebilir. Bu kendi isteğine bağlı olduğu için extend ile ifade edilir ve sınıfa kaydolma işlemine bağlanır.

  Ek bilgi (derste anlatılmadı): UML'de iki ilişki kesikli okla çizilir ve okların yönü farklıdır. include okunda ok, temel senaryodan her zaman çalıştırılan senaryoya gider (sınıfa kaydol → sınıf bilgisini görüntüle). extend okunda ok, isteğe bağlı senaryodan temel senaryoya gider (özel ders seç → sınıfa kaydol).

Örnek diyagramda aktörlerin işlemleri şöyledir:

- **Üye:** Üye olur, sınıf bilgisini görüntüler, sınıfa kaydolur (include: sınıf bilgisini görüntüle), isterse özel ders seçer (extend), ödeme yapar.
- **Eğitmen:** Katılımcıları görür, bilgilerini günceller.
- **Yönetim:** Sınıf oluşturur.

Bu diyagramlar projeyi daha iyi anlamayı ve kavramayı sağlar, çünkü aktörler ve yapabilecekleri işlemler net biçimde görülür.

4.3 Veri Akış Diyagramı

Veri akış diyagramı (DFD), elde edilen bilgiler doğrultusunda sistemdeki verilerin nasıl taşındığını modeller. Seviye seviye tasarlanır: Önce kaba bir taslak yapılır, sonra ayrıntılandırılır.

Seviye 0. Sistem bir dörtgen içinde tek parça olarak gösterilir, buna "spor salonu sistemi" denir. Üç aktörle arasındaki veri akışı kabaca çizilir. Üye tarafında akışlar şöyledir:

- Üye, üye olmak için kişisel bilgilerini sisteme verir.
- Sınıf seçimi için bilgi girer.
- Sınıf bilgileri üyeye geri döner, yani üye sınıf bilgilerini görüntüler.
- Üye, ücret ödeyerek aylık ücretini öder.

Yönetim ve eğitmenle de benzer veri akışları vardır.

Seviye 1. Spor salonu sisteminin içindeki işlemler açılır. İki örnek incelendi:

- **Üye olma:** Üye, kişisel bilgilerini "üye olma" işlemine verir. Bu işlem üye bilgilerini "üyeler" adlı veri tabanı tablosunda saklar.
- **Sınıf oluşturma:** Yönetim sınıf bilgilerini girerek "sınıf oluşturma" işlemini başlatır. İşlem aldığı bilgileri "sınıf bilgisi" veri tabanı tablosunda saklar.

İki örnekte de bilgiler bir yerden işleme gelir ve işlemden bir yere aktarılır.

> **Hocanın vurgusu:** Veri akış diyagramında her işlemin mutlaka bir girdisi ve bir çıktısı olmalıdır. Bunu unutmayın.

Seviye 2. Bir işlemin altı ayrıntılandırılır. Bu, üye olma, ödeme yapma, sınıfa kaydolma gibi her işlem için yapılabilir. Derste ödeme yapma örneği alındı. Ödeme yapmanın altında şu alt işlemler vardır:

1. Toplam ücretin hesaplanması. Çıkan "toplam ücret" sonraki işleme geçer.
2. Ödemenin alınması. Çıkan "ödenen tutar" sonraki işleme geçer.
3. Kalan borcun hesaplanması.

4.4 Veri Sözlüğü

Veri sözlüğü, diyagramdaki bir veri akışında neyin, nereden nereye gittiğini yazar. Derste iki kayıt örneği verildi:

Alan	Kişisel bilgiler	Sınıf bilgileri
İçerik	Üye olacak kişinin vermesi gereken ad, soyad, yaş, boy, kilo, iletişim bilgileri	Sınıfta verilen ders, eğitmen, katılımcı bilgileri, takvim bilgileri
Kaynak	Üye	Altyazıda geçmiyor
Hedef	Üye olma işlemi	Altyazıda geçmiyor
Veri yapıları	Üye detayları	Altyazıda geçmiyor
Miktar	Altyazıda geçmiyor	Günlük ortalama 10 giriş

Bu kayıt, kişisel bilgiler akışında nelerin üye olma adımına aktarıldığını gösterir. Aynı şekilde ücret için de bir kayıt oluşturulur: Hangi bilgiler gidiyor, nereden gidiyor, nereye gidiyor? Bu her veri akışı için yapılabilir.

4.5 Varlık-İlişki Diyagramı

Varlık-ilişki diyagramının (Entity Relationship Diagram, ERD) gösterim kuralları şunlardır:

- **Varlık:** Dikdörtgenle gösterilir.
- **Nitelik:** Elipsle gösterilir ve varlığa bağlanır.
- **İlişki:** Varlıklar arasında eşkenar dörtgenle gösterilir.

> **Ek bilgi (derste anlatılmadı):** Dikdörtgen, elips ve eşkenar dörtgen kullanan bu gösterim Chen gösterimi diye bilinir.

Örnekte "Üye" varlığının nitelikleri yaş, boy, kilo, iletişim bilgisi ve ad-soyaddır. "Sınıf" varlığının nitelikleri isim, yer, tarih, saat ve kapasitedir. Üye bir sınıfa kaydolduğu için aralarında **kaydolma** ilişkisi vardır.

İlişkilere de özellik eklenebilir. Kaydolma ilişkisinin "kayıt tarihi" özelliği olabilir. Asistan, şemalarda ilişkilere genelde çok fazla özellik eklenmediğini, ama bunun mümkün olduğunun unutulmaması gerektiğini söyledi.

Diyagramdaki ilişkiler şunlardır:

Varlıklar	İlişki
Üye - Sınıf	Kaydolma
Üye - Muhasebe	Ödeme yap
Muhasebe - Yönetim	Raporlama
Yönetim - Sınıf	Sınıf oluşturma
Sınıf - Eğitmen	Ders verme

> **Ders sonrası not:** Asistan ER diyagramındaki "Üye" için "sınıf", özellikleri için "attribute" dedi. ER modelinde doğru terimler varlık (entity) ve niteliktir (attribute). "Sınıf" UML sınıf diyagramının terimidir.

> **Ders sonrası not:** Asistan veri akış diyagramını gereksinim analizinin "son adımı" diye tanıttı (00:28:21), ama ardından yine bu bölümde veri sözlüğünü ve varlık-ilişki diyagramını anlattı. Dersin akışına göre son adım bunlardır.

5. Sistem Tasarımı

Sistem tasarımı, sistem mimarisini ortaya koymak amacıyla programdaki yapısal öğeler arasındaki ilişkileri belirler. Bu bölümde üç diyagram anlatıldı: yapı diyagramı, sınıf diyagramı ve ardışıl diyagram.

5.1 Yapı Diyagramı

Yapı diyagramı, ana modül ile ara birimler arasındaki veri akışını ayrıntılı ve hiyerarşik olarak ortaya koyar. Önce ana modüle bağlı alt modüller belirlenir. İlerleyen süreçte birbirinden ayrı olduğu anlaşılan modüller, ayrı diyagramların ana modülü olarak ifade edilebilir.

Örnekte ana modül spor salonudur. Bağlı alt modüller arasında üye olma, üye giriş yapma ve sınıfa kaydolma vardır. Modüller arasında geçen bilgiler oklarla işaretlenir.

Ok uçlarındaki yuvarlaklar. Üye olma modülünde üyelik bilgileri spor salonuna aktarılır, sonra spor salonundan üyelik onayı gelir. İki okun ucundaki yuvarlaklar farklıdır: Birinin içi boş, birinin içi doludur. Asistan bu farkı sordu ve kendisi açıkladı. Üye olmada bilgi aktarılıyor, ardından bir onay geliyor. Onaylarda ve doğrulamalarda genellikle içi dolu yuvarlak kullanılır, çünkü yalnızca bir onay mesajı gönderilir. Benzer şekilde üye giriş yapmada kullanıcı bilgileri spor salonuna gider ve burada doğrulanır, bunun gösterimi de farklıdır.

> **Ek bilgi (derste anlatılmadı):** Yapı diyagramında içi boş yuvarlaklı ok veri çifti (data couple), yani modüller arasında taşınan veridir. İçi dolu yuvarlaklı ok kontrol bayrağıdır (control flag), yani onay, hata gibi durum bilgisidir.

Eşkenar dörtgen. Sınıfa kaydol modülünün altında grup dersine katıl ve özel derse katıl modülleri vardır. Buradaki eşkenar dörtgen bir seçeneği ifade eder: Sınıfa kaydolan üye grup derslerine de özel derslere de katılabilir.

5.2 Sınıf Diyagramı

Nesneye dayalı yaklaşım benimsendiği için sistem tasarımında bir sınıf diyagramı oluşturmak gerekir. Örnekte bir spor salonu yönetimi ve onun altında eğitmen, sınıf, üye, üyelik ve ders sınıfları vardır.

Diyagramdaki sınıflar şunlardır:

- **Spor Salonu Yönetimi:** `borcOgren()`
- **Üye:** `uyeNo`, `isim`, `yas`, `boy`, `kilo`
- **Üyelik:** `uyeOl(Uye)`
- **Sınıf:** `id`, `ders`, `takvim`, `egitmen`, `uyeler: List<Uye>`, `getSinifBilgisi()`
- **Eğitmen:** `id`, `isim`, `branslar: String[]`
- **Ders (soyut):** `isim`, `saatUcreti: double`, `getSaatUcreti()`

Kalıtım. Aerobik, Dans ve Pilates sınıfları, Ders sınıfının özelliklerini kalıtımla alır. Ders soyut (abstract) bir sınıftır. Asistan soyut sınıfın özelliğini hatırlattı: Soyut sınıfın doğrudan nesnesi oluşturulamaz.

İlişkiler. Asistan sınıftan ilişkileri yorumlamasını istedi, yanıt gelmeyince kendisi açıkladı:

- **Eğitmen - Sınıf:** Bir eğitmen birden fazla sınıfa katılabilir, yani ders verebilir.
- **Üye - Üyelik:** Bir üyenin bir tane üyeliği olabilir.
- **Üye - Sınıf:** Asistan bunu "zayıf varlık ilişkisi" diye adlandırdı ve "bir üye varsa bir sınıfa kayıtlı olması gerekir" diye açıkladı.

  Ders sonrası not: Asistan Üye-Sınıf ilişkisine "zayıf varlık ilişkisi" dedi; burada üç ayrı kavram karışmıştır. Kısa nottaki kayda göre diyagramda bu ilişki içi boş elmasla çizilmiştir ve bu UML'de birleştirmedir (aggregation). "Her üye en az bir sınıfa kayıtlı olmalı" kuralı bir katılım kısıtıdır (zorunlu katılım). Zayıf varlık ise ER modelinde kendi anahtarı olmayan ve sahip varlığa bağlı olarak tanımlanan varlıktır.

Nesneye yönelik programlama yapılacaksa bu diyagram önemlidir. Projede yapılacak bütün işlemler ayrıntılı olarak görülür ve programlama doğrudan bu diyagram üzerinden yürütülür.

5.3 Ardışıl (Sequence) Diyagram

Ardışıl diyagram, üye olma, sınıfa kaydolma ve ücret görüntüleme gibi işlemlerde hangi metot çağrısının ne şekilde gerçekleştiğini gösterir. Örnekte Üye sınıfına ait "üye" isimli bir nesne vardır.

Gösterim. Giden mesajlar düz çizgiyle, geri dönen mesajlar kesikli çizgiyle gösterilir. Asistan diyagramdaki dikdörtgenlerin ne anlama geldiğini sordu ve kendisi açıkladı: Dikdörtgenler süreçleri, örneğin üye ol mesajı ile onay mesajı arasında geçen süreyi ifade eder.

> **Ders sonrası not:** Asistanın "süreç" dediği dikdörtgenlerin UML'deki adı aktivasyon (execution occurrence) çubuğudur. Nesnenin bir işlemi yürüttüğü süreyi gösterir.

Diyagramdaki akışlar şunlardır:

- **Üye olma:** Üye nesnesi, üyelik bilgilerini parametre alan üye ol fonksiyonunu çağırır. Spor salonu bir onay mesajı döner.
- **Sınıf bilgisi alma:** Mesaj Sınıf sınıfına gider, sınıf bilgisi geri döner.
- **Sınıfa kaydolma:** Üyelik bilgileri yine Sınıf sınıfına gider.
- **Borç öğrenme:** Aşağıdaki adımlarla yürür.

Borç öğrenme akışında bir mesaj zincir gibi sınıflar arasında ilerler ve sonuç aynı yoldan geri döner:

1. Üye, spor salonuna borç öğren mesajı gönderir.
2. Spor salonu, ödenecek tutarı almak için Sınıf'ı çağırır (`getOdenecekTutar`).
3. Sınıf, ders saati ücretini almak için Ders sınıfına gider.
4. Ders, saat ücretini Sınıf'a döndürür.
5. Sınıf, katılım süresini hesaplar ve tutarı bulur.
6. Tutar spor salonuna, oradan üyeye döndürülür.

Tutarın hesabı şöyledir:

$$\text{Ödenecek tutar} = \text{saat ücreti} \times \text{katılım süresi}$$

Sınıf diyagramında yapılacak işlemler sınıfların altına zaten yazılmıştı. Ardışıl diyagram bunun üstüne şunu ekler: Üye bir işlem yapmak istediğinde bilgilerin nereden nereye aktarılacağı, geri dönüşlerin nasıl alınıp üyeye iletileceği ayrıntılı olarak görülür. Böylece nasıl programlanacağı ve fonksiyonların birbirine nasıl geri dönüş sağlayacağı netleşir.

6. Değerlendirme Tablosu

Son adımda, projeye göre fizibilite çalışması, gereksinim analizi ve sistem tasarımı önem sırasına göre puanlanarak bir değerlendirme yapılabilir. Asistan bu örnekte 100 üzerinden bir puanlama yaptı. Bu tablo, bir projede adımlara verilen önem ağırlığının örneğidir.

Adım	Puan
Teknik fizibilite	8
Ekonomik fizibilite	7
Zaman fizibilitesi	7
Sosyal fizibilite	2
Yönetim fizibilitesi	3
Yasal fizibilite	3
Fizibilite çalışması toplamı	**30**
Gereksinim analizi	40
Sistem tasarımı	30
Genel toplam	**100**

Doğrulama: Fizibilite toplamı $8 + 7 + 7 + 2 + 3 + 3 = 30$, genel toplam $30 + 40 + 30 = 100$ çıkar.

> **Ders sonrası not:** Sözlü anlatımda puanlar "yasal 3, yönetim sosyal 2 gibi" diye geçti. Bu sözlü ifadedeki gibi yönetim de 2 alınırsa fizibilite toplamı $8 + 7 + 7 + 2 + 2 + 3 = 29$, genel toplam $29 + 40 + 30 = 99$ olur. Tablodaki yönetim 3 değeriyle toplam 100'e tamamlanır.

Puanlar projeden projeye değişir. Böyle bir tablonun yararı, projede nelerin öncelikli olduğunu göstermesidir. Asistanın örneği şöyleydi: Projeye ayrılan bütçe çok yüksekse ekonomik fizibilitede harcanacak bütçe o kadar önemli olmayabilir. Teknik fizibilite öne çıkar. Daha iyi donanım, sistemin daha hızlı çalışması için birden fazla sunucu, kişisel verilerin ve şirket içi bilgilerin korunması için bir güvenlik hizmeti istenebilir.

7. Özet ve Diyagram Araçları

Asistan örnek projenin özetini şöyle verdi. Teknik ihtiyaç olarak sunucu, router ve masaüstü bilgisayar raporlandı. Yazılım olarak Java ve JavaScript, yazılım geliştirme modeli olarak şelale modeli belirlendi. Kullanım senaryosu, veri akış, sınıf, yapı, ardışıl ve ER diyagramları oluşturuldu. Spor salonu sisteminin gereksinimlerini belirlemek için fizibilite çalışması yapıldı ve gerçekleştirme için gerekli kaynaklar belirtildi.

> **Ders sonrası not:** Asistan özette bu altı diyagramın hepsini "sistem tasarımında" oluşturulmuş gibi saydı (00:45:57). Dersin akışına göre kullanım senaryosu, veri akış ve ER diyagramları gereksinim analizinde, yapı, sınıf ve ardışıl diyagramlar sistem tasarımında çizildi.
>
> **Hocanın vurgusu:** Yalnızca örneğe odaklanmayın. ER, kullanım senaryosu ve veri akış diyagramları önemlidir, çünkü bunları kendi projelerinizde kullanacaksınız.

Araçlar. Asistan, MS Project'te Gantt ve PERT oluşturulabildiğini, diğer diyagramların hepsinin oluşturulabileceğinden emin olmadığını söyledi. UML diyagramları için hem çevrim içi sistemler hem de bilgisayara indirilebilen programlar vardır. Gantt diyagramı için de farklı programlar kullanılabilir.

Dersin Toparlaması

Bu uygulamada aynı örnek, analiz ve tasarımın üç aşamasından geçirildi. Fizibilite çalışmasında altı tür üzerinden alternatif sistemler üretildi ve matrisle karşılaştırılarak başlangıç için yeterli olan tek sunuculu Sistem 1 seçildi. Gereksinim analizinde anketle veri toplandı, aktörlerin işlemleri kullanım senaryosu dokümanı ve diyagramıyla, verinin hareketi seviye seviye veri akış diyagramı ve veri sözlüğüyle, varlıklar ise ER diyagramıyla modellendi.

Sistem tasarımında bu modeller programa yaklaştırıldı. Yapı diyagramı modül hiyerarşisini, sınıf diyagramı programın iskeletini, ardışıl diyagram ise metot çağrılarının sırasını gösterdi. Son olarak değerlendirme tablosu, bu adımların ağırlığının projenin önceliklerine göre değiştiğini hatırlattı. Dersten alınacak temel ders, örneğin kendisinden çok bu adımların ve diyagramların kendi projelerinize aktarılmasıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.